GENERATIVE AI & LLMS

# Large Language Models (GPT & LLaMA)

A field guide to how GPT and LLaMA actually work — from attention and tokens to RLHF, context windows, sampling, inference cost, and the open-vs-closed decision.

LLM | Transformer | GPT | LLaMA | RLHF | fine-tuning | inference

Study Pack · Comprehensive reference & interview prep

CONTENTS


1. What LLMs Are and Why They Matter
2. Setup and Getting Started
3. Tokenization: How Text Becomes Numbers
4. The Transformer Architecture
5. Pretraining vs Fine-Tuning vs Alignment
6. Context Windows and the KV Cache
7. Sampling: Controlling the Output
8. GPT vs LLaMA: Families, Open vs Closed
9. Inference Cost and Optimization
10. Common Patterns and a Worked Mini-Example
11. Cheat-Sheet and Comparison Reference
12. Best Practices and Common Pitfalls
13. Limitations and Hallucination
14. Interview Questions & Answers
15. Further Learning


## What LLMs Are and Why They Matter

A Large Language Model (LLM) is a neural network — almost always a Transformer — trained on enormous text corpora to predict the next token. From that single objective emerges fluent generation, translation, reasoning, coding, and tool use. GPT and LLaMA

are the two archetypal families: one closed and API-first, one open-weight and self-hostable.

At their core, modern LLMs are **autoregressive next-token predictors**. Given a sequence of tokens, the model outputs a probability distribution over the vocabulary for the next token, samples one, appends it, and repeats. Everything else — chat, summarization, agents — is this loop plus clever prompting and post-training.

> **Mental model**
> Think of an LLM as a gigantic, differentiable autocomplete. It has no database of facts and no execution engine inside; it has *weights* that encode statistical structure of language. "Knowledge" is lossy compression of the training set into those weights.

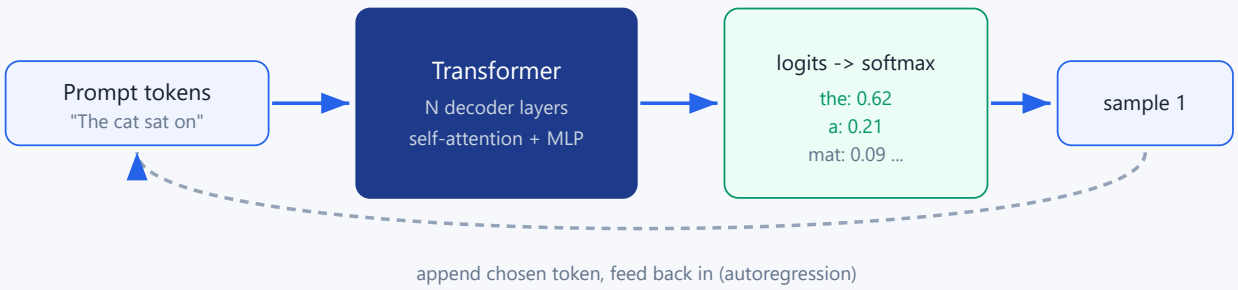



*Fig 1. The autoregressive generation loop: predict a distribution, sample, append, repeat.*

Why they matter: a single pretrained model generalizes across tasks it was never explicitly trained for (**in-context learning**). This collapses what used to be dozens of bespoke NLP models into one general-purpose system you steer with natural language.

## Setup and Getting Started

You interact with LLMs two ways: hosted **APIs** (closed models like GPT) or **local weights** (open models like LLaMA). Below are both.

### HOSTED API (OPENAI-STYLE)

BASH

```
# Install the official SDK
pip install openai

# Set your key (never hard-code it)
export OPENAI_API_KEY="sk-..."
```

PYTHON

```python
from openai import OpenAI

client = OpenAI()  # reads OPENAI_API_KEY from the environment

resp = client.chat.completions.create(
    model="gpt-4o",
    messages=[
        {"role": "system", "content": "You are a terse assistant."},
        {"role": "user", "content": "Explain attention in one sentence."},
    ],
    temperature=0.7,
    max_tokens=120,
)
print(resp.choices[0].message.content)
```

### LOCAL OPEN-WEIGHT MODEL (LLAMA VIA HUGGING FACE)

BASH

```
pip install "transformers≥4.44" accelerate torch
# Access to Meta LLaMA weights requires accepting the license on the model page.
huggingface-cli login
```

PYTHON

```python
import torch
from transformers import AutoModelForCausalLM, AutoTokenizer

name = "meta-llama/Llama-3.1-8B-Instruct"
tok = AutoTokenizer.from_pretrained(name)
model = AutoModelForCausalLM.from_pretrained(
    name, torch_dtype=torch.bfloat16, device_map="auto"
)

msgs = [{"role": "user", "content": "Give me a haiku about tensors."}]
# Chat templates insert the correct special tokens for this model family.
ids = tok.apply_chat_template(msgs, add_generation_prompt=True, return_tensors="pt").t
out = model.generate(ids, max_new_tokens=64, do_sample=True, temperature=0.8, top_p=0.
print(tok.decode(out[0][ids.shape[-1]:], skip_special_tokens=True))
```

**Fast local serving**

For a one-command local chat server, `ollama run llama3.1` downloads a quantized GGUF and exposes an OpenAI-compatible endpoint on `localhost:11434`. Great for prototyping without GPUs.

## Tokenization: How Text Becomes Numbers

Models do not see characters or words — they see **tokens**, integer IDs for subword fragments produced by a **Byte-Pair Encoding (BPE)**-style tokenizer. Common words are single tokens; rare words split into pieces.

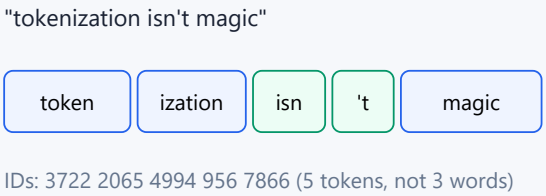



*Fig 2. Subword tokenization splits uncommon strings; whitespace is usually attached to the following token.*

PYTHON

```python
import tiktoken  # OpenAI's BPE tokenizer

enc = tiktoken.get_encoding("o200k_base")  # GPT-4o vocabulary
ids = enc.encode("tokenization isn't magic")
print(ids)            # list of integer token ids
print(len(ids))       # token count → this is what you pay for
print([enc.decode([i]) for i in ids])  # round-trip each piece
```

> ⚠ **Tokens != words**
>
> English averages ~0.75 words per token (~4 chars/token). Code, non-Latin scripts, and long numbers tokenize far less efficiently. Always measure cost and context limits in *tokens*, never characters.

## The Transformer Architecture

GPT and LLaMA are both **decoder-only** Transformers. A stack of identical layers transforms token embeddings into context-aware representations; a final linear head projects to vocabulary logits.

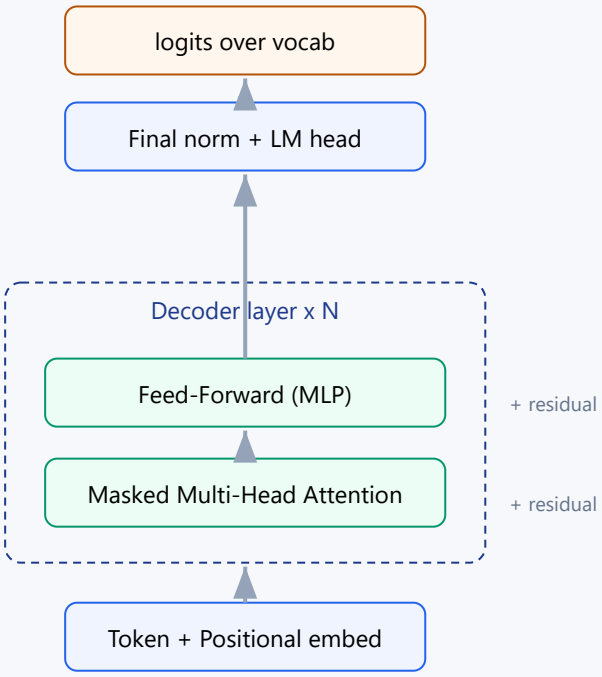



*Fig 3. Decoder-only block: each layer applies masked self-attention then an MLP, both wrapped in residual connections and normalization.*

## Self-Attention: the core operation

Each token builds a **query**, and compares it against every other token's **key** to decide how much of each token's **value** to pull in. The causal mask prevents attending to future tokens.

The scaled dot-product attention is: $\text{Attention}(Q,K,V) = \text{softmax}(QK^T / \sqrt{d_k})\,V$.

PYTHON

```python
import torch, torch.nn.functional as F

def causal_self_attention(x, Wq, Wk, Wv):
    # x: (seq, d_model)
    Q, K, V = x @ Wq, x @ Wk, x @ Wv       # (seq, d_k)
    d_k = Q.shape[-1]
    scores = (Q @ K.transpose(-2, -1)) / d_k**0.5   # (seq, seq)
    # causal mask: token i may not look at j > i
    mask = torch.triu(torch.ones_like(scores), diagonal=1).bool()
    scores = scores.masked_fill(mask, float("-inf"))
    attn = F.softmax(scores, dim=-1)        # rows sum to 1
    return attn @ V                          # weighted blend of values
```

> **Why "multi-head"**
> Instead of one attention, the model runs *h* parallel heads on lower-dimensional projections, each free to specialize (syntax, coreference, position). Outputs are concatenated and mixed. This is cheap parallelism that adds representational richness.

## What GPT and LLaMA changed from the original Transformer

| Component | Original (2017) | Modern GPT / LLaMA |
|---|---|---|
| Structure | Encoder-decoder | Decoder-only |
| Positional info | Absolute sinusoidal | RoPE (rotary) — LLaMA; learned/absolute — early GPT |
| Normalization | Post-LayerNorm | Pre-norm; RMSNorm in LLaMA |
| Activation | ReLU | SwiGLU / GeGLU |
| Attention (serving) | Full MHA | Grouped-Query Attention (GQA) to shrink KV cache |

# Pretraining vs Fine-Tuning vs Alignment

A deployed chat model goes through distinct stages. Confusing them is a common interview trap.

**Pretraining** -> **SFT** -> **RLHF / DPO** -> **Deployed model**

### 1. PRETRAINING

Self-supervised next-token prediction on trillions of tokens of web text, code, books. Produces a *base model* with broad knowledge but no instruction-following manners. The expensive stage (millions of GPU-hours).

### 2. SUPERVISED FINE-TUNING

Train on curated (prompt, ideal-response) pairs so the base model learns the chat/instruction format. Cheap relative to pretraining; produces an *instruct* model.

### 3. PREFERENCE ALIGNMENT

RLHF or DPO nudges the model toward responses humans prefer — helpful, honest, harmless. Shapes *style and safety*, not raw knowledge.

## RLHF and its simpler cousin DPO

**RLHF** (Reinforcement Learning from Human Feedback) trains a *reward model* on human comparisons of pairs of answers, then optimizes the LLM against that reward with PPO, keeping a KL penalty so it does not drift far from the SFT model. **DPO** (Direct Preference Optimization) skips the separate reward model and PPO loop, optimizing a closed-form loss directly on preference pairs — simpler and now very common.

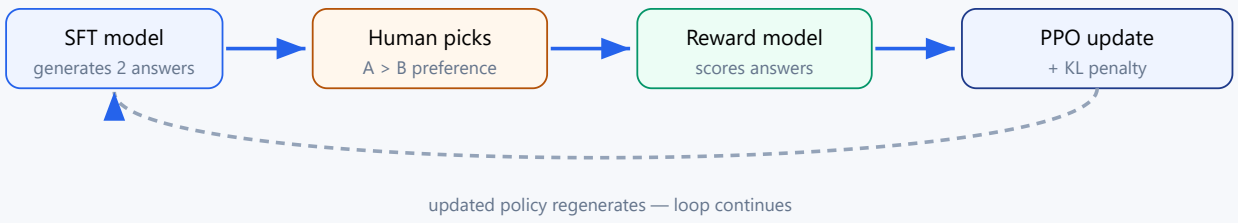



*Fig 4. The RLHF loop. DPO collapses the reward-model + PPO stages into a single supervised-style objective on the preference pairs.*

### Parameter-efficient fine-tuning (LoRA)

Full fine-tuning of a large model is costly. **LoRA** freezes the base weights and learns small low-rank adapter matrices, cutting trainable parameters by 100-1000x. **QLoRA** adds 4-bit quantization so a 7B-13B model fine-tunes on a single consumer GPU.

PYTHON

```python
from peft import LoraConfig, get_peft_model

cfg = LoraConfig(
    r=16, lora_alpha=32, lora_dropout=0.05,
    target_modules=["q_proj", "v_proj"],  # inject adapters into attention
    task_type="CAUSAL_LM",
)
model = get_peft_model(base_model, cfg)
model.print_trainable_parameters()  # e.g. 0.1% of total params are trainable
```

## Context Windows and the KV Cache

The **context window** is the maximum number of tokens (prompt + generation) a model can attend to at once — 8K for early LLaMA, 128K for GPT-4o, up to 1M+ for frontier long-context models in 2026. It bounds how much you can stuff into a single call.

During generation the model caches each layer's keys and values for past tokens — the **KV cache** — so it does not recompute them every step. This makes decoding fast but memory-hungry: KV memory grows linearly with sequence length and is often the real serving bottleneck.

> ⚠ **Quadratic cost**
> Attention compute scales as $O(n^2)$ with sequence length n. Doubling context roughly quadruples attention FLOPs. Long-context models rely on tricks (GQA, FlashAttention, sliding-window/sparse attention) to stay tractable.

| Term | Meaning | Practical impact |
|---|---|---|
| Context window | Max total tokens per call | Hard cap on prompt + output |
| KV cache | Stored keys/values of past tokens | GPU memory grows with length |
| Prefill | Processing the input prompt | Compute-bound, parallel |
| Decode | Generating tokens one by one | Memory-bandwidth-bound, serial |

## Sampling: Controlling the Output

At each step you have a distribution over the vocabulary. **Decoding strategy** decides which token to actually emit — the knob between deterministic and creative.

### TEMPERATURE

Divides logits before softmax. `T<1` sharpens (safer, repetitive); `T>1` flattens (diverse, riskier); `T=0` is greedy/argmax.

### TOP-K

Keep only the k highest-probability tokens, renormalize, then sample. Caps the tail of unlikely tokens.

### TOP-P (NUCLEUS)

Keep the smallest set whose cumulative probability exceeds p (e.g. 0.9). Adapts the candidate count to the distribution's shape.

### REPETITION / FREQUENCY PENALTY

Down-weight tokens already seen to reduce loops and verbatim repetition.

PYTHON

```python
import torch, torch.nn.functional as F

def sample(logits, temperature=0.8, top_p=0.9):
    logits = logits / max(temperature, 1e-6)
    probs = F.softmax(logits, dim=-1)
    # nucleus (top-p) filtering
    sorted_probs, idx = torch.sort(probs, descending=True)
    cum = torch.cumsum(sorted_probs, dim=-1)
    keep = cum - sorted_probs <= top_p          # keep until mass exceeds p
    sorted_probs = sorted_probs * keep
    sorted_probs /= sorted_probs.sum()
    choice = torch.multinomial(sorted_probs, 1)
    return idx[choice]                          # map back to vocab id
```

✦ **Rule of thumb**

Factual extraction, classification, code edits: `temperature=0` for reproducibility. Brainstorming, creative writing: `0.7–1.0` with `top_p=0.9`. Avoid setting both high temperature and high top-p/top-k if you need coherence.

## GPT vs LLaMA: Families, Open vs Closed

The two families embody the industry's central tension: **closed, hosted, frontier** capability versus **open-weight, self-hostable** control.

| Dimension | GPT family (OpenAI) | LLaMA family (Meta) |
|---|---|---|
| Access | Closed — API only, weights private | Open-weight — download, run, fine-tune |
| Hosting | Vendor cloud | Your hardware or any cloud |
| Customization | Fine-tuning via API, prompting | Full fine-tuning, LoRA, quantization, surgery |
| Cost model | Per-token usage billing | Upfront GPU / infra cost |
| Data control | Data leaves your boundary | Fully on-prem possible |
| Typical edge | Peak frontier capability, tooling | Privacy, cost at scale, no lock-in |

> **"Open" nuance**
> LLaMA is *open-weight*, not fully open-source: Meta publishes weights under a community license (with some use restrictions) but not the training data or full pipeline. Truly open efforts (e.g. OLMo) release data and code too. In interviews, say "open-weight" — it signals precision.

### Choosing between them

#### REACH FOR CLOSED (GPT)

You want the highest capability now, minimal ops burden, strong tool/function-calling, and token-based cost is acceptable. Fast to ship.

#### REACH FOR OPEN (LLAMA)

Data must stay on-prem, you need predictable cost at high volume, deep customization, offline/edge deployment, or freedom from vendor changes.

## Inference Cost and Optimization

Serving dominates lifetime cost. Two levers: reduce the model's footprint and reduce per-request waste.

#### QUANTIZATION

Store weights in 8-bit or 4-bit instead of 16-bit. 4x smaller, modest quality loss. GGUF/AWQ/GPTQ formats make LLaMA run on laptops.

#### BATCHING

Continuous/in-flight batching (vLLM, TGI) packs many requests through the GPU together, hugely improving throughput and $/token.

#### CACHING

Prompt/prefix caching reuses the KV cache for shared system prompts. Speculative decoding uses a small draft model to propose tokens a big model verifies.

BASH

```bash
# High-throughput open-model serving with vLLM (OpenAI-compatible API)
pip install vllm
vllm serve meta-llama/Llama-3.1-8B-Instruct \
  --max-model-len 8192 --dtype bfloat16 --quantization awq
```

✦ **Cost intuition**

You pay for input (prompt) and output tokens, usually at different rates, with output tokens typically costing more because decoding is serial. Trimming system prompts, truncating RAG context, and capping `max_tokens` are the cheapest wins.

## Common Patterns and a Worked Mini-Example

Most production LLM value comes from a few repeatable patterns.

### RAG

Retrieve relevant chunks from a vector store, inject into the prompt. Grounds answers in your data and reduces hallucination.

### FUNCTION / TOOL CALLING

Model emits structured JSON requesting a tool; your code runs it and feeds results back. Foundation of agents.

### STRUCTURED OUTPUT

Constrain generation to a JSON schema for reliable downstream parsing.

### FEW-SHOT PROMPTING

Show 2-5 examples in the prompt to steer format and behavior without any training.

#### MINI-EXAMPLE: A GROUNDED Q&A OVER YOUR OWN DOCS (RAG)

PYTHON

```python
from openai import OpenAI
client = OpenAI()

# 1) Pretend retrieval: top chunks from a vector search for the user query.
chunks = [
    "Refunds are issued within 14 days of purchase.",
    "Digital goods are non-refundable once downloaded.",
]
context = "\n".join(f"- {c}" for c in chunks)

user_q = "Can I get a refund on a downloaded ebook after 3 days?"

resp = client.chat.completions.create(
    model="gpt-4o",
    temperature=0,  # deterministic for a policy answer
    messages=[
        {"role": "system", "content":
         "Answer ONLY from the context. If unsupported, say you don't know."},
        {"role": "user", "content": f"Context:\n{context}\n\nQuestion: {user_q}"},
    ],
)
print(resp.choices[0].message.content)
# → Grounded answer citing the non-refundable-digital-goods rule.
```

# Cheat-Sheet and Comparison Reference

| Concept | One-line definition |
|---|---|
| Token | Subword unit the model reads/writes; billing + context unit |
| Embedding | Dense vector representation of a token or text |
| Attention | Weighted lookup letting each token read others |
| Context window | Max tokens per call (prompt + output) |
| Temperature | Randomness knob for sampling |
| Top-p | Nucleus sampling cutoff on cumulative probability |
| SFT | Supervised fine-tuning on instruction pairs |
| RLHF / DPO | Align outputs to human preferences |
| LoRA / QLoRA | Cheap low-rank (and quantized) fine-tuning |
| RAG | Retrieve external docs into the prompt |
| Hallucination | Confident but false generated content |
| Quantization | Lower-precision weights for cheaper inference |

| Decoding setting | Deterministic | Creative |
|---|---|---|
| temperature | 0.0 | 0.8 - 1.0 |
| top_p | 1.0 (unused) | 0.9 |
| Use for | Extraction, code, classification | Writing, ideation |

# Best Practices and Common Pitfalls

✓ **Do**

Put stable instructions in the system prompt; measure everything in tokens; set `temperature=0` when you need reproducibility; ground factual tasks with RAG; validate structured output against a schema; log prompts and versions for evals.

✦ **Do**

Pin a specific model version in production, keep a regression eval set, and A/B before upgrading — model updates can silently change behavior.

⚠ **Watch out**

Context limits are shared by input and output — a huge prompt can starve the response. Lost-in-the-middle: models attend less to content buried mid-context, so put the most important instructions at the start or end.

✕ **Never**

Trust an LLM for facts, math, or citations without verification or tools — it will hallucinate fluently and confidently. Never paste secrets into third-party APIs you have not vetted. Never assume output is safe to execute or render without sanitization (prompt injection is real).

# Limitations and Hallucination

Understanding failure modes is as important as capabilities — and a frequent interview focus.

### HALLUCINATION

The objective rewards plausible *continuations*, not truth. With no grounding the model fabricates confidently. Mitigate with RAG, tool use, citations, and lower temperature.

### KNOWLEDGE CUTOFF

Weights freeze at training time; the model is unaware of later events unless given tools/retrieval.

### WEAK SYMBOLIC REASONING

Multi-step arithmetic and strict logic are error-prone. Offload to code/tools; chain-of-thought helps but is not a guarantee.

### PROMPT INJECTION

Untrusted text in the context can hijack instructions. Treat retrieved/user content as data, not commands; isolate privileges.

**Bias & cost**

Models reflect biases in training data, are non-deterministic by default, and are computationally expensive. These are engineering constraints to design around, not bugs to be fully "fixed."

## Interview Questions & Answers

**Why are modern LLMs decoder-only instead of encoder-decoder?**

Pure next-token prediction with a causal mask scales cleanly, uses one unified stack, and lets the same model do generation, few-shot learning, and (via prompting) tasks that once needed an encoder. The simpler architecture trains and serves more efficiently at scale.

**Explain self-attention in one paragraph.**

Each token projects into a query, key, and value. The query dot-products with every key to score relevance, scaled by √d_k and softmaxed into weights; the output is the weighted sum of values. A causal mask blocks attending to future tokens. This lets every token gather context from the whole sequence in parallel.

**What is the difference between pretraining, SFT, and RLHF?**

Pretraining is self-supervised next-token prediction on raw text — it builds knowledge. SFT fine-tunes on curated instruction/response pairs to teach the chat format. RLHF (or DPO) aligns outputs to human preferences for helpfulness and safety. Knowledge mostly comes from pretraining; manners and style come from the later stages.

**What does temperature do, and when would you set it to 0?**

Temperature scales logits before softmax, controlling randomness. Lower sharpens the distribution; 0 makes decoding greedy/deterministic. Use 0 for extraction, classification, code, or anywhere you need reproducible, factual output.

**Why does doubling the context window more than double the cost?**

Self-attention is $O(n^2)$ in sequence length, so attention compute roughly quadruples when n doubles, and the KV cache memory grows linearly, often becoming the serving bottleneck. That is why long-context models use GQA, FlashAttention, and sparse/sliding-window attention.

### What is the KV cache and why does it matter?

During autoregressive decoding, each layer's keys and values for past tokens are cached so they aren't recomputed every step. It makes generation fast but its memory grows with sequence length and batch size, frequently limiting how many concurrent requests a GPU can serve.

### How does LoRA make fine-tuning cheap?

It freezes the base weights and injects small trainable low-rank matrices (A·B) into chosen layers, so only a tiny fraction of parameters update. QLoRA adds 4-bit quantization of the frozen base, letting mid-size models fine-tune on a single consumer GPU with minimal quality loss.

### Open-weight vs closed models — how do you choose?

Closed (GPT) gives peak capability and low ops burden with per-token billing. Open-weight (LLaMA) gives data control, on-prem/offline deployment, deep customization, and predictable cost at high volume. Choose on data-privacy requirements, volume economics, customization needs, and tolerance for vendor lock-in.

### Why do LLMs hallucinate and how do you reduce it?

The training objective rewards plausible continuations, not verified truth, so with no grounding the model fills gaps confidently. Reduce it with retrieval-augmented generation, tool/function calling, requiring citations, lower temperature, and asking the model to abstain when context is insufficient.

### What is the difference between top-k and top-p sampling?

Top-k keeps a fixed number k of highest-probability tokens; top-p (nucleus) keeps the smallest set whose cumulative probability exceeds p, so the candidate count adapts to how peaked or flat the distribution is. Top-p generally handles varied distributions more gracefully.

### What is prompt injection and how do you defend against it?

It's when untrusted text (user input or retrieved documents) contains instructions that hijack the model's behavior. Defend by treating external content as data not commands, separating trusted system instructions, least-privilege tool access, output validation, and never auto-executing model output without checks.

### How does RLHF differ from DPO?

RLHF trains a separate reward model from human preference comparisons, then optimizes the policy against it with PPO plus a KL penalty. DPO reparameterizes the problem into a single

> supervised-style loss directly on preference pairs — no reward model or RL loop — making it simpler, more stable, and cheaper while often matching RLHF quality.

## Further Learning

- Read the original **"Attention Is All You Need"** (2017) and the GPT-2/GPT-3 papers for the scaling story.
- Study the **LLaMA** technical reports for the modern decoder recipe (RoPE, RMSNorm, SwiGLU, GQA).
- Implement a tiny Transformer from scratch (Karpathy's **nanoGPT / minGPT**) to internalize attention and training.
- Build a small **RAG app** with a vector DB, then add tool/function calling and structured output.
- Fine-tune an open model with **LoRA/QLoRA** on a domain dataset; serve it with **vLLM** and measure throughput.
- Set up an **eval harness** (golden set + LLM-as-judge) so you can safely compare and upgrade models.
- Explore agent frameworks and the **Model Context Protocol** to connect LLMs to real tools and data.